Studio Nx

JS 03 — safeNextPath()

2026-10-12

JS 03 — `safeNextPath(next, origin)`

항목	내용
파일	`studionx/frontend/static/js/core/auth.js`
작업	P1-02 프론트 공통 JS (PRD 01 FR-105)
작성	Claude (AI), 2026-10-06, 사용자 검토
이전 · 다음	02 `apiFetch()` · 04 `loginUrl()`

1. 하는 일

로그인한 뒤 돌아갈 주소(`next`)가 **이 사이트 안의 경로**인지 검사합니다. 안전하면 그대로 돌려주고, 아니면 `null`을 돌려줍니다.

2. 언제 쓰나

로그인이 필요한 페이지(마이페이지, 내 프로젝트, 회원 정보, 시퀀서)에 로그인하지 않고 들어오면 `home.html#login?next=/myPage.html`로 보냅니다. 로그인 모달에서 로그인에 성공하면 `next`로 돌아갑니다.

상황	부르는 곳
주소의 `#login?next=…`를 읽을 때	`parseLoginHash()`(05)를 거쳐 `home.js`의 로그인 모달에서 씁니다
로그인에 성공한 뒤 이동하기 직전	로그인 폼 처리(P1-07)에서 한 번 더 확인합니다

막는 공격: 열린 리다이렉트 (open redirect)

`next`는 주소창에 그대로 들어 있어서 **누구나 바꿀 수 있습니다**. 검사가 없으면 다음과 같은 일이 생깁니다.

1. 공격자가 `https://studionx…/home.html#login?next=https://evil.com/fake-login` 링크를 보냅니다.
2. 사용자는 진짜 Studio Nx 주소라고 믿고 로그인합니다.
3. 로그인 직후 가짜 사이트로 이동하고, "비밀번호를 다시 입력하세요" 같은 피싱 화면을 보게 됩니다.

그래서 **같은 사이트의 경로만** 허용합니다.

3. 코드

import (파일 맨 위)

```
import { api, clearAccessToken, getAccessToken, refreshAccessToken } from './api.js';
```

`auth.js` 맨 위의 import 줄입니다. **이 함수는 import한 것을 쓰지 않습니다.** `URL`과 `location`은 브라우저의 기본 기능입니다.

함수

```
export function safeNextPath(next, origin = location.origin) {
  if (typeof next !== 'string' || !next.startsWith('/')) return null;
```

```
  if (next.startsWith('//') || next.includes('\\')) return null;
  try {
    return new URL(next, origin).origin === origin ? next : null;
  } catch {
    return null;
  }
}
```

문법 풀이

import 문법	뜻
`import { a, b } from './api.js'`	`api.js`가 `export`한 것 중 `a`와 `b`를 이름 그대로 가져옵니다. `'./'`는 같은 폴더라는 뜻입니다. 브라우저는 `<script type="module">`일 때만 `import`를 읽습니다
`export` (함수 앞)	다른 파일이 `import { 함수 }`로 가져다 쓸 수 있게 내보냅니다. 없으면 이 파일 안에서만 쓸 수 있습니다

식	뜻
`origin = location.origin`	기본값 매개변수입니다. 보통은 지금 사이트(`http://localhost:8000`)이고, 시험할 때만 다른 값을 넣습니다
`typeof next !== 'string'`	문자열이 아니면(`null`, `undefined`, 숫자) 거부합니다
`'\'`	자바스크립트 문자열 안에서 역슬래시 한 글자 `\`를 뜻합니다
`new URL(next, origin)`	`next`를 `origin` 기준으로 해석한 완전한 주소입니다. `new URL('/a', 'http://x')`는 `http://x/a`가 됩니다
`.origin === origin`	해석한 주소의 `프로토콜://호스트:포트`가 이 사이트와 같은지 확인합니다
`try … catch`	이상한 문자열 때문에 `new URL`이 오류를 내면 `null`을 돌려줍니다

4. 흐름

1. 문자열이 아니거나 `/`로 시작하지 않으면 거부합니다. `https://…`, `javascript:…`, `myPage.html`이 여기서 걸립니다.
2. `//`로 시작하거나 `\`가 들어 있으면 거부합니다. 브라우저는 이런 값을 **다른 사이트의 주소**로 읽습니다.
3. 마지막으로 브라우저와 같은 방식(`new URL`)으로 해석해서 이 사이트인지 확인합니다.

5. 왜 이렇게

선택	이유
`/`로 시작하는 경로만 허용	가장 단순하고 확실한 기준입니다. 전체 주소(`http://…`)는 이 사이트의 주소여도 거부합니다. 화면은 경로만 넘기기 때문입니다
`//evil.com` 거부	"프로토콜 상대 주소"라서 브라우저는 `https://evil.com`으로 이동합니다
`\`가 들어가면 거부	브라우저는 `/\evil.com`의 `\`를 `/`로 바꿔서 `//evil.com`으로 읽습니다
`new URL`로 한 번 더 확인	위의 규칙이 놓친 특이한 형태도 실제 해석 결과로 막습니다
오류 대신 `null`을 돌려줌	부르는 쪽은 `null`이면 기본 페이지로 보내면 됩니다

6. 시험

입력	결과
`/myPage.html`, `/studioNx.html?projectId=3`, `/myProject.html#new`	그대로 돌려줍니다
`//evil.com`, `/\evil.com`	`null`
`https://evil.com/x`, `http://localhost:8000/myPage.html`	`null` (전체 주소는 모두 거부합니다)
`javascript:alert(1)`, `myPage.html`, `''`	`null`
`null`, `undefined`, `42`	`null`
`/%2F%2Fevil.com`	그대로 돌려줍니다. 해석하면 `http://localhost:8000/%2F%2Fevil.com`(이 사이트 안의 경로)이라서 안전합니다